MEDS Lab - Module 4

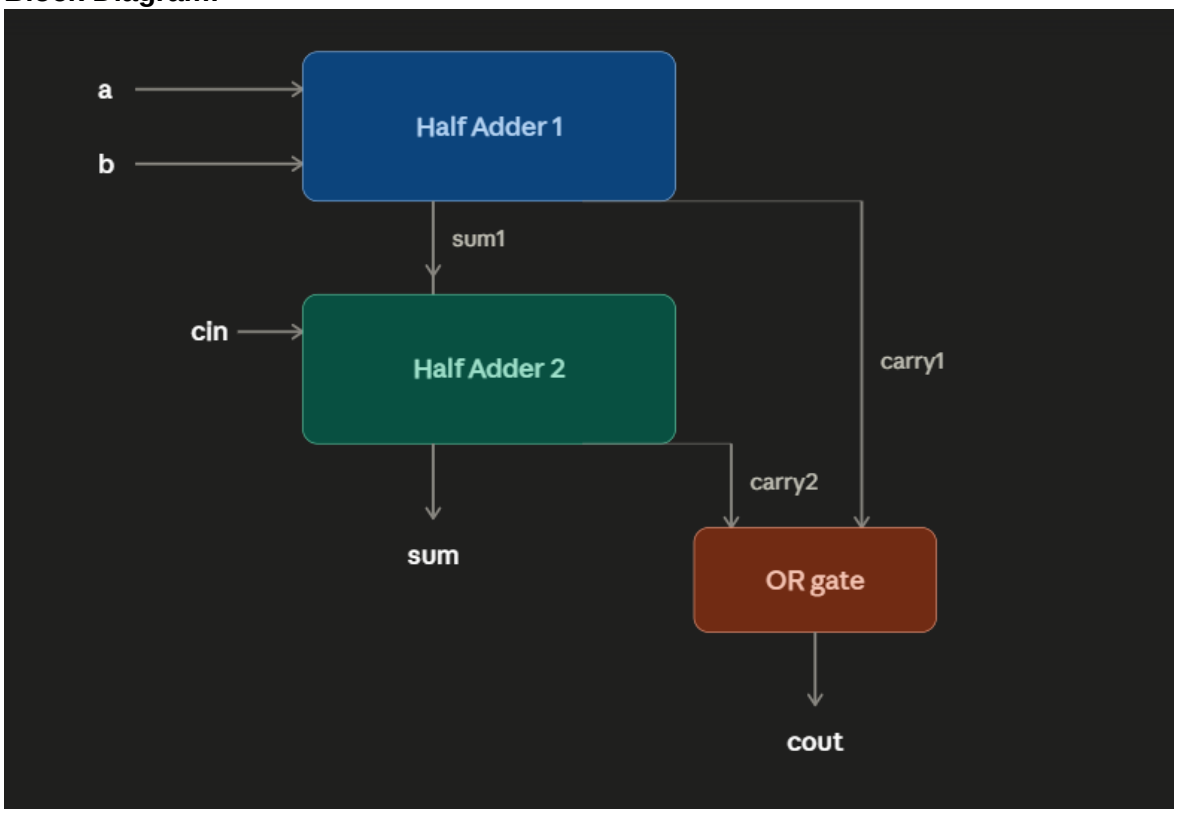
Section 6 Report: Combinational Design Practices

Name: Muhammad Kaleem Faryad
University: UET Lahore

Task 3: Half Adder and Full Adder

1. **EDA Playground Link:** https://edaplayground.com/x/qFdK

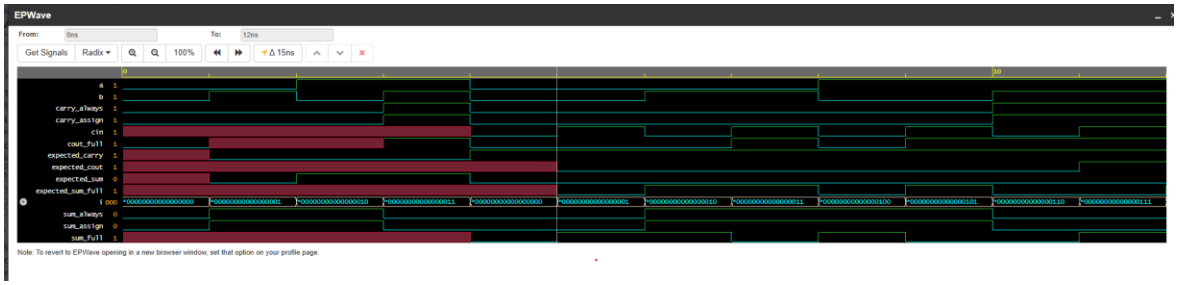
2. **Block Diagram:**



3. **Simulation Output Screenshot:**

```
 HALF ADDER TEST
[PASS] Assign SUM (00) Value=0
[PASS] Assign CARRY (00) Value=0
[PASS] Always SUM (00) Value=0
[PASS] Always CARRY (00) Value=0

[PASS] Assign SUM (01) Value=1
[PASS] Assign CARRY (01) Value=0
[PASS] Always SUM (01) Value=1
[PASS] Always CARRY (01) Value=0

[PASS] Assign SUM (10) Value=1
[PASS] Assign CARRY (10) Value=0
[PASS] Always SUM (10) Value=1
[PASS] Always CARRY (10) Value=0

[PASS] Assign SUM (11) Value=0
[PASS] Assign CARRY (11) Value=1
[PASS] Always SUM (11) Value=0
[PASS] Always CARRY (11) Value=1

 FULL ADDER TEST
[PASS] SUM (000) Value=0
[PASS] COUT (000) Value=0

[PASS] SUM (001) Value=1
[PASS] COUT (001) Value=0

[PASS] SUM (010) Value=1
[PASS] COUT (010) Value=0

[PASS] SUM (011) Value=0
[FAIL] COUT (011) Expected=0 Actual=1

[PASS] SUM (100) Value=1
[PASS] COUT (100) Value=0

[PASS] SUM (101) Value=0
[FAIL] COUT (101) Expected=0 Actual=1

[PASS] SUM (110) Value=0
[PASS] COUT (110) Value=1

[PASS] SUM (111) Value=1
```

4. **Waveform Screenshot:**



5. **Explanation:** The half adder was implemented using two different coding styles. The first implementation used continuous assignment (assign) statements, while the second implementation used an always_comb block with procedural assignments. Although the syntax of the two implementations is different, both describe the same Boolean equations and therefore synthesize into identical hardware. The full adder was then implemented structurally by connecting two half adders and one OR gate. The first half adder generated the intermediate sum and carry. The second half adder added the carry-in bit to the intermediate sum. Finally, an OR gate combined both carry outputs to produce the final carry-out signal. Simulation results confirmed that both half adder implementations produced identical outputs for every input combination, and the full adder generated the correct sum and carry outputs.

Task 4: NOR-Only Realization

1. **EDA Playground Link:** https://edaplayground.com/x/mBVs

2. Boolean Algebra

F = (B+C+D)(A′+B+C)(A′+D)

Using De Morgan's theorem,

P=(B+C+D)′ , Q=(A'+B+C)' , R=(A′+D)′

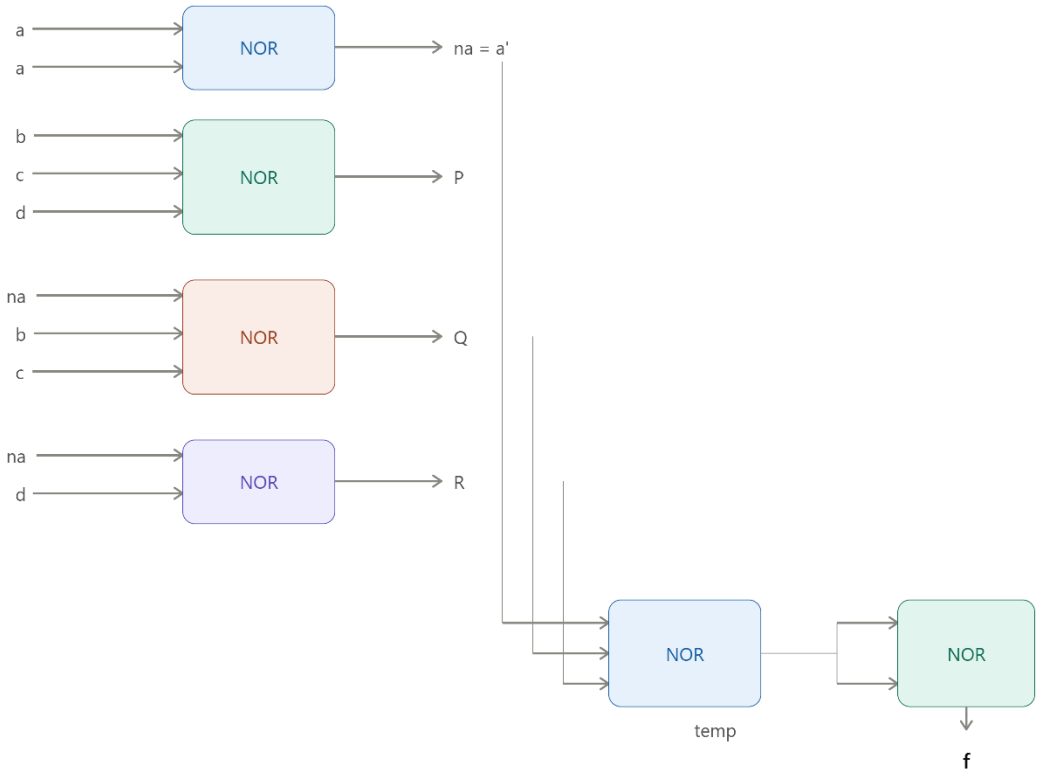
Then

F=((P+Q+R)')′

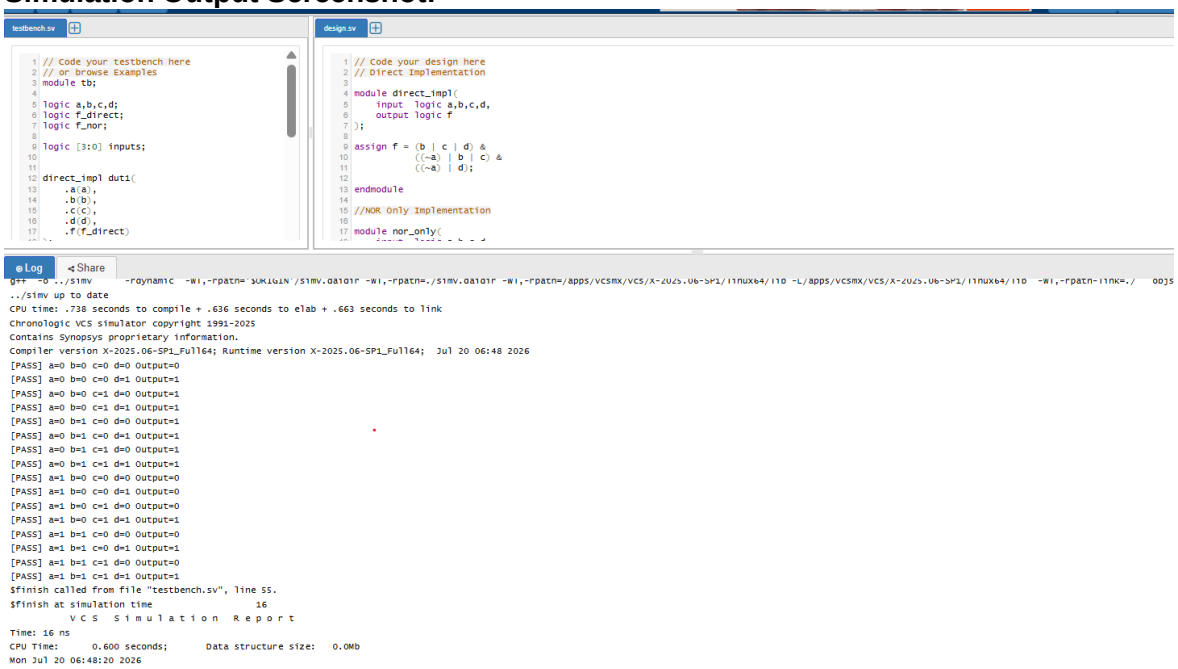
Since

A′=(A+A)′

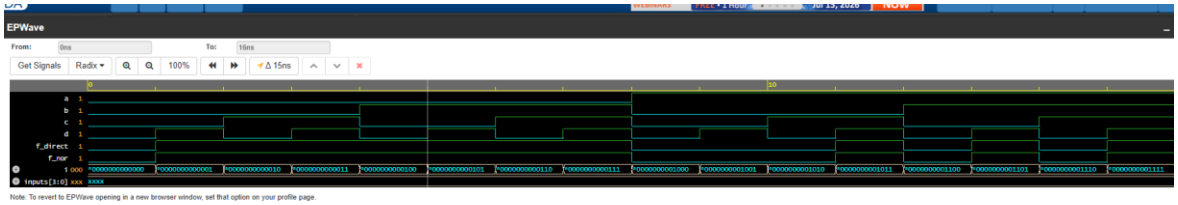
3. Block Diagram:



4. Simulation Output Screenshot:



5. Waveform Screenshot:

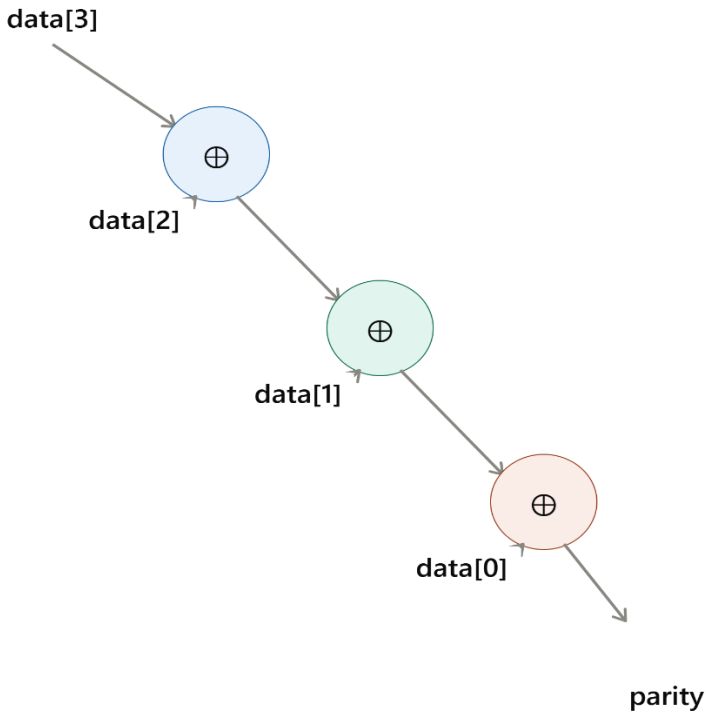


6. Explanation: The given Boolean expression was converted into a NOR-only implementation using De Morgan's theorem. Each complemented expression was generated using NOR gates, including the inversion of input A by connecting both inputs of a NOR gate together. A second module implemented the original Boolean expression directly, while another module implemented the equivalent NOR-only circuit. A self-checking testbench exhaustively tested all sixteen possible input combinations and automatically compared both outputs. Every test case passed successfully, confirming that both implementations are logically equivalent.

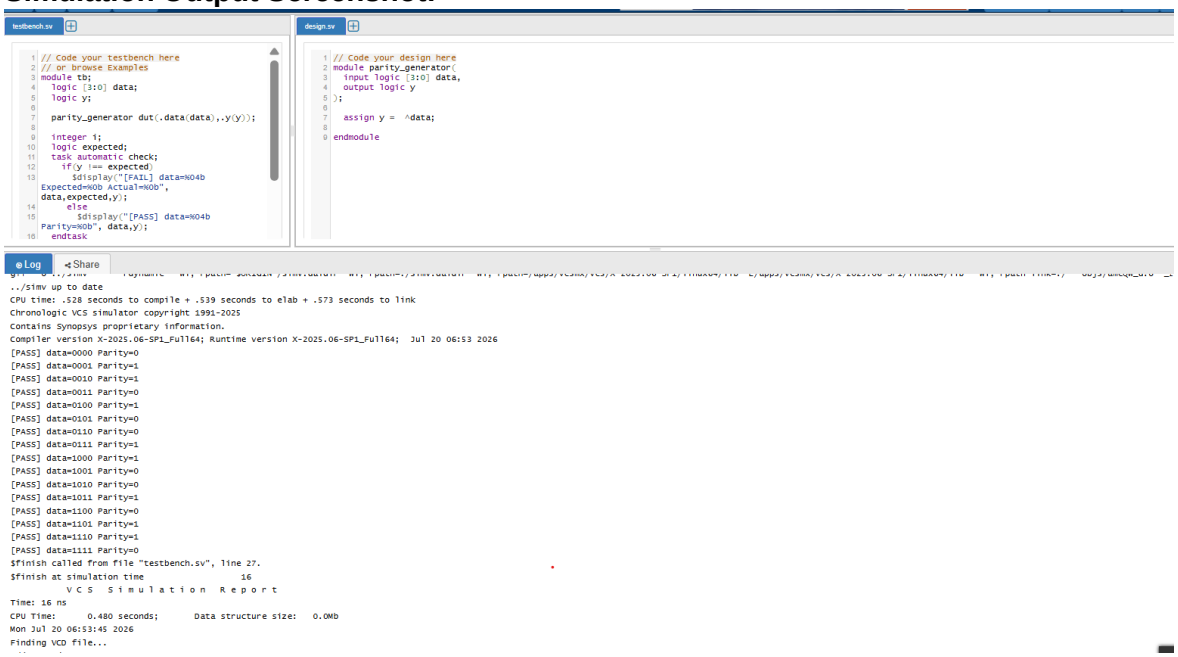
Task 5: NOR-Only Realization

1. **EDA Playground Link:** https://edaplayground.com/x/j6PP

2. **Block Diagram:**



3. **Simulation Output Screenshot:**



testbench.sv

```
// Code your testbench here
// or browse Examples
module tb;
  logic [3:0] data;
  logic y;

  parity_generator dut(.data(data),.y(y));

  integer i;
  logic expected;
  task automatic check;
    if(y !== expected)
      $display("[FAIL] data=%04b
Expected=%0b Actual=%0b",
data,expected,y);
    else
      $display("[PASS] data=%04b
Parity=%0b", data,y);
  endtask
```

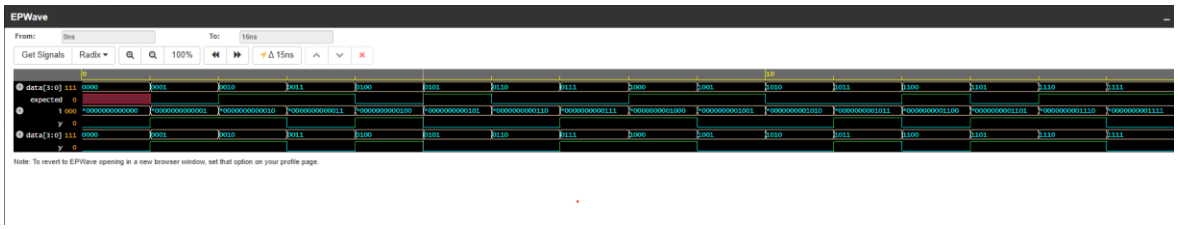
design.sv

```
// Code your design here
module parity_generator(
  input logic [3:0] data,
  output logic y
);

  assign y =  ^data;

endmodule
```

Log | Share

```
../simv up to date
CPU time: .528 seconds to compile + .539 seconds to elab + .573 seconds to link
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64;  Jul 20 06:53 2026
[PASS] data=0000 Parity=0
[PASS] data=0001 Parity=1
[PASS] data=0010 Parity=1
[PASS] data=0011 Parity=0
[PASS] data=0100 Parity=1
[PASS] data=0101 Parity=0
[PASS] data=0110 Parity=0
[PASS] data=0111 Parity=1
[PASS] data=1000 Parity=1
[PASS] data=1001 Parity=0
[PASS] data=1010 Parity=0
[PASS] data=1011 Parity=1
[PASS] data=1100 Parity=0
[PASS] data=1101 Parity=1
[PASS] data=1110 Parity=1
[PASS] data=1111 Parity=0
$finish called from file "testbench.sv", line 27.
$finish at simulation time                16
           V C S   S i m u l a t i o n   R e p o r t
Time: 16 ns
CPU Time:      0.480 seconds;       Data structure size:   0.0Mb
Mon Jul 20 06:53:45 2026
Finding VCD file...
```

4. **Waveform Screenshot:**



5. **Explanation:** The given Boolean expression was converted into a NOR-only implementation using De Morgan's theorem. Each complemented expression was generated using NOR gates, including the inversion of input A by connecting both inputs of a NOR gate together.A second module implemented the original Boolean expression directly, while another module implemented the equivalent NOR-only circuit. A self-checking testbench exhaustively tested all sixteen possible input combinations and automatically compared both outputs. Every test case passed successfully, confirming that both implementations are logically equivalent.